Version of Record: https://www.sciencedirect.com/science/article/pii/S221313371730032X
Manuscript_cee18aa210e95629f213eed2d7b3f5a7

Faster Catalog Matching on Graphics Processing Units

Matthias A. Lee[a], Tamás Budavári[b]

[a]*Department of Computer Science, Johns Hopkins University, Baltimore, MD, 21218 USA.*
[b]*Department of Applied Math and Statistics, Johns Hopkins University, Baltimore, MD, 21218 USA.*

Abstract

One of the most fundamental problems in observational astronomy is the cross-identification of sources. Observations are made at different times in different wavelengths with separate instruments, resulting in a large set of independent observations. The scientific outcome is often limited by our ability to quickly perform associations across catalogs. The matching, however, is difficult scientifically, statistically as well as computationally. The former two require detailed physical modeling and advanced probabilistic concepts; the latter is due to the large volumes of data and the problem's combinatorial nature. In order to tackle the computational challenge and to prepare for future surveys we developed a new implementation on Graphics Processing Units. Our solution scales across multiple devices and can process hundreds of trillions of crossmatch candidates per second in a single machine.

Keywords: methods: statistical — astrometry — catalogs — surveys

1. Introduction

Modern telescopes produce vast volumes of data every night. With the current and upcoming advances in technology, survey data-sets are growing at a tremendous pace. To maximize the scientific value of each experiments we often need to combine their observations with other surveys. The identification of objects across multiple catalogs and surveys can lead to new discoveries and breakthroughs but the speed of matching is a limiting factor when combining the large outputs of many experiments. A number of studies have focused on the statistical aspect of this challenge (Budavári and Szalay, 2008, Heinis et al., 2009, Kerekes et al., 2010, Budavári and Loredo, 2015) but they all rely on fast 2-way matching engines to find candidate associations first.

The current solutions including the SkyQuery (Dobos et al., 2012, Budavári et al., 2013) and the CDS X-Match Service (Boch et al., 2016) use hierarchical indexes and space-filling curves to accelerate the process (Kunszt et al., 2001, Górski et al., 2005). While their performance is great they are far from the speed of interactive data exploration, the ability to do on-the-fly catalog federation would be a game-changer. To illustrate the big-data aspect of the project, lets consider a relatively small scenario of matching GALEX (50 million objects) with SDSS DR7 (150 million objects), a naïve implementation would require 7.5 quadrillion (10^{15}) comparisons.

In this paper we describe a novel approach that takes advantage of the extreme parallelism available on modern GPUs as well as an efficient method for reducing the total number of comparisons. The tool we present can crossmatch at rates of over a trillion candidate pairs per millisecond. We will limit our investigation to matching only two catalogs but without assuming that they are sorted or indexed in any way ahead of time. This choice is motivated by the fact that n-way associations can be built up by 2-way matching using the best guess direction of the partial matches (Budavári and Szalay, 2008).

2. Divide and Conquer

The combinatorial scaling of a naïve matching approach can be remedied by quickly eliminating pairs at large separations. This is often achieved by partitioning the sky and considering only nearby areas instead of the entire sphere. These heuristic algorithms often rely on hierarchical division schemes such as Igloo (Crittenden, 2000), Hierarchical Triangular Mesh (HTM; Kunszt et al., 2001), HEALPix (Górski et al., 2005) or SDSSPix (Scranton et al.).

2.1. Building on the Zones Algorithm

Our approach follows the division scheme of the *Zones Algorithm* of Gray et al. (2007), which employs a much simpler scheme. It breaks up the sky into constant declination rings called the *zones*; see Figure 1. All zones have the same *height*, h, measured in angle. For details on choosing a good value for h, see Section 3. A zone identifier is assigned to each source i based on its declination $\delta_i \in [-90°, 90°]$ as given by

$$Z_i = \left\lfloor \frac{\delta_i + 90°}{h} \right\rfloor , \tag{1}$$

Email address: matthiaslee@jhu.edu (Matthias A. Lee)

where $\lfloor \rfloor$ indicates the floor function rounding down to the closest integer. Sources with the same values are in the same zones, which creates easy and computationally cheap division of sources across catalogs. The simplicity of this equation enables the immediate implementation in any language unlike the more complicated schemes listed above. This also fits well with the indexing facilities of relational database management systems (RDBMS). For example, the SkyQuery solution relies on zones for the largest matching problems, as an optimal query plan can essentially stream through the data on the hard drive with high efficiency and little memory overhead. The key for crossmatching is to use the same zone layout across all catalogs and eliminate all zone pairs that are farther than a specified search radius. The resulting set of zone pairs effectively mitigates any overlap needed to account for positional errors. Within the zones it can be beneficial to further sort by the right-ascension of the objects to quickly eliminate candidates that are too far apart even before calculating the separation between sources.

2.2. Layers of Parallelism

Modern GPUs can run thousands of threads in parallel. In addition they also have superior memory bandwidth as compared to CPUs. The capacity of RAM, however, is somewhat more limited than that of today's servers. With these parameters in mind, we design our architecture to take maximum advantage of multiple GPUs in a single box. We will further assume that one of the catalogs, the smaller, can fit in the computer's memory, which will increase the speed of the execution. This is not, however, a significant constraint considering that 1 billion sources can be stored in 24GB of memory with 8-byte numbers for object identifier and the two celestial coordinates.

We slice the input catalogs into suitable *segments* that fit on the GPUs and build a job scheduler to process pairs of these segments from two catalogs. For the purposes of these next sections, let us consider two unsorted catalogs, `Catalog_A` and `Catalog_B`, each catalog consisting of objects identified by their *ObjId* (64bit integer), *RA* (double) and *Dec* (double). Our method breaks down into two main levels of parallelism. At the high level we distribute the problem across GPUs and at the lower level we parallelize across the many-core architecture within each GPU.

2.2.1. Preparing the Segments

At runtime we begin the loading process of each catalog by subdividing each into multiple segments of size n. The size is chosen such that two segments, plus the overhead needed for processing, can fit into GPU memory. While GPU memory itself is very fast, transfers to and from are comparatively slow and hence should be kept to a minimum. The division of the catalogs into segments is purely a construct allowing us to manage the data more effectively in order to fit the data onto the GPU and minimize GPU-memory transfer overhead.

Input : Catalogs A and B, each containing objects identified by (id, ra, dec) and (id', ra', dec'), respectively

gpu-foreach *Segment of A* **do**
- Load Segment from disk;
- Copy to GPU;
- Sort by Zones and RA;
- Identify Zone boundaries;
- Copy back from GPU;

end

while *Segments in B remain* **do**
- **gpu-foreach** *Segment of B per GPU* **do**
  - Load Segment from disk;
  - Copy to GPU;
  - Sort by Zones and RA;
  - Identify Zone boundaries;
  - Copy back from GPU;
- **end**
- **gpu-foreach** *Segment-Segment pair* **do**
  - Compute distance metric for every object within each Zone-Zone pairing;
  - Accumulate comparison results;
- **end**

end

Output: List of Object-Object pairings (id, id'), where (ra, dec) and (ra', dec') are considered a match

Algorithm 1: High-level Crossmatch Processing Steps

We begin by loading the smaller of the two input catalogs, into CPU memory, segmenting it as we go. After each segment is read from disk, it is loaded onto a GPU and sorted by the zone identifiers Z and right-ascension using the C/C++ CUDA *Thrust* library (Bell and Hoberock, 2011). This is done via a custom comparator implemented as a functor which on-the-fly and in parallel calculates the Z values. Arithmetics on the GPU are very fast and repeated calculations of the zone identifier does not slow down the process as we save on memory transfer, which is the typical bottleneck. Sorting by zone identifiers is only part of the battle, we also need to identify the zone boundaries, enabling us to easily separate out zones at execution time. This has also been implemented in parallel using the `thurst::lower_bound()` and `thrust::upper_bound()` search functions, which are based on Thrust's vectorized binary search.

We then loop over the larger catalog, reading one segment per available GPU. These segments are also loaded onto the GPUs and sorted by zones. At this point the preparation has taken place and the system is ready to loop through the segments of the catalog loaded in CPU memory.

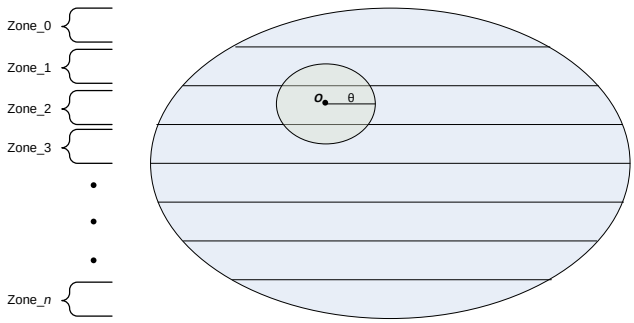

Figure 1: The Zones Algorithm: the coordinate plane is subdivided along one axis into small strips of height h called *zones*. To find an object within a search radius, θ, all zones are searched which overlap with search radius.

2.2.2. Jobs and Workers

A dynamic execution environment is implemented where a pool of worker threads, one thread per GPU, wait for jobs consisting of two segments, one from each input catalog. A *job manager* keeps track of the current memory content of each GPU and assigns the jobs for processing. This is done via a greedy algorithm that performs close to the ideal in realistic settings for 2-way matching. Each job is a unique pairing of one segment from each catalog, such that every segment of `Catalog_A` is paired once with every segment of `Catalog_B`. The scheduling of jobs is also optimized, such that preference is given to jobs which share a common segment with the previous job, allowing us to avoid unnecessary overhead of reloading the same segment multiple times.

Each worker independently computes the crossmatch between the two segments applying the Zones Algorithm to only consider relevant pairs of zones given the search radius. A custom CUDA kernel (Xmatch) is launched for each pair of zones, which is the heart of the implementation.

2.2.3. Matching on the GPU

A typical modern GPU features thousands of cores arranged on multiple Streaming Multiprocessors (SM). For example, the NVIDIA Tesla K20c device has 13 SMs with 2496 cores in total, which can run 26624 threads (2048 threads per SM) in parallel at any given time. To map sets of GPU threads across the available SMs, threads are arranged in a hierarchy of `blocks` and `grids`, where a `blocks` can be thought of as an array of threads and `grids` are an arrangement of `blocks`. Both can be 1-, 2-, or 3-dimensional arrays as appropriate for the applications. The notion of a `block` is important as the threads within are guaranteed to run on the same SM and hence can communicate very efficiently. It is important to note that each core has access to different types of memory with vast differences in access speeds. For example, shared memory – directly part of the SM, – can be accessed in only 11-22 clock cycles, where as global memory takes 200-800 clock cycles (access cycles dependent on hardware generation). The SM automatically attempts to hide much of the global memory access latency by swapping out warps which are currently waiting for memory transfers to return. Additionally global memory accesses should be coalesced, meaning memory will perform best when all threads in a warp access a contiguous block of memory. This allows the GPU to perform a single memory transaction, instead of separately fetching memory for each thread. More in-depth information on these topics can be found in Nvidia (2011).

Our implementation takes these optimizations into account wherever possible. We launch kernels such that every source-source distance calculation is assigned to a separate but suitably arranged GPU-thread allowing for coalesced memory accesses for parallel read and write operations, which are often the capital expense in many-core applications. Indeed, finding the nearby pairs in the sorted `zones` is very fast when run on thousands of threads in parallel. The challenge is to efficiently save the results. The custom `xmatch` kernel computes all the pairwise distances and flags the candidate associations as a boolean output into a shared memory array. Naturally, this is very sparse, making it impractical for saving. To optimize memory utilization and therefore to minimize the overhead of extra data transfers, we use a series of parallel algorithms within the `block` to find matched candidates. The implementation details are beyond the scope of this paper but we note that at the core of the algorithm are parallel prefix scan (Harris et al., 2007) and bisection algorithms that convert to a sparse representation within shared memory where only the indices of the candidate associations are saved. At the end of this procedure the same threads that previously found the candidates, are used to make a coalesced write to copy the results to global memory for ultimately returning to CPU memory. We repeat these steps for each zone-zone comparison, after which all matches are copied back from the GPU and written out to results files.

2.3. Available from C++ and Python

Our code is available publicly under the MIT open source license on Github[1]. We welcome any feedback and pull requests. The codebase mainly consists of a `Command line tool` and a `Python interface`. The `Command line tool` is implemented in C++/CUDA, reading a simple binary format directly available from most databases: simply `ObjID`, `RA` and `Dec` columns, 8 bytes each, totaling 24 bytes per object. We also provide a small python utility to convert CSV files to and from this format. The minimal argument set consists of 3 required arguments, `catalog_A`, `catalog_B` and `segmentSize`, all other arguments have reasonable defaults. The optimal segment size should be set such that two segments plus about 15% overhead can fit into GPU memory. The full command-line options are

[1] http://matthiaslee.github.io/Xmatch (Budavari and Lee, 2013)

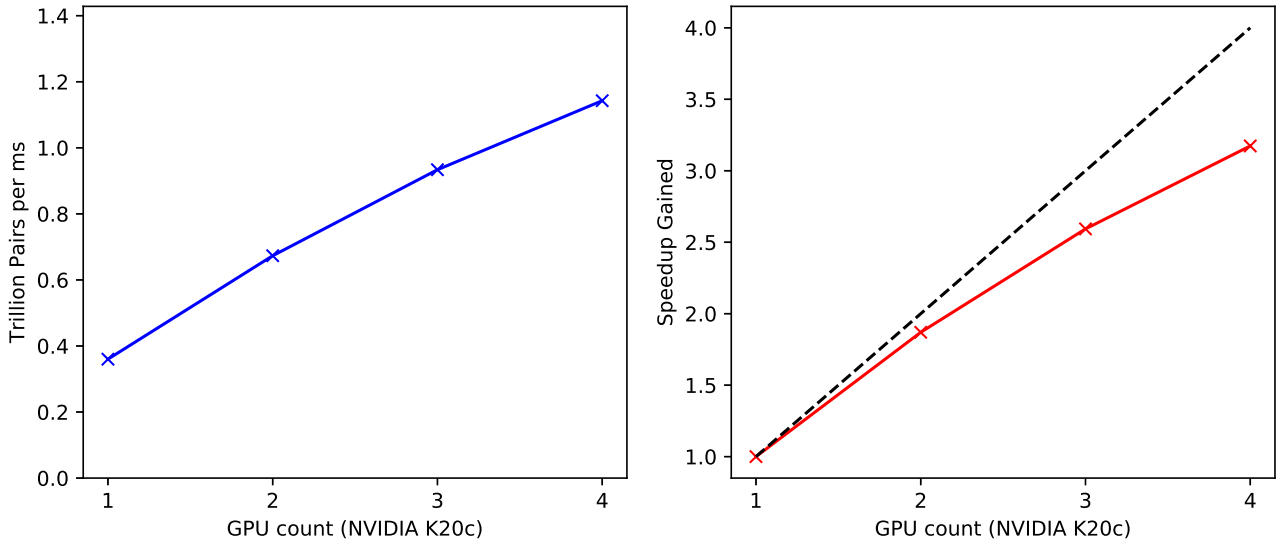

Figure 2: Same-task performance characteristics of matching two catalogs of 450 million each, using a zone height of 2 arcseconds and a search radius of 1.5 arcseconds. *left:* Matching rate scaling in trillion candidate pairs per milliseconds based on the number of GPUs. *right:* Speedup gained by scaling up the number of GPUs. The dashed line indicates perfectly linear scaling.

shown in Usage 1. The `Python interface` provides direct access to the C++/CUDA-native function using Python's ctypes interface. The exposed interface offers an identical function-set to the command line tool. For example, the Python lines

```
>>> from Xmatch import crossmatch
>>> crossmatch('catalogA.idradec',
               'catalogB.idradec',
                segSize=100000000,
                radius=1.5,
                zoneHeight=1.5,
                numGPU=6)
```

would work with the binary catalog files to produce the matches as expected.

3. Performance

Our tool can compare two catalogs of 150M objects in approx. 1.5 minutes and two catalogs of 450M objects in approx. 9 minutes on a single NVIDIA k20c GPU. As we scale up the number of GPUs, we see much improved performance. With four NVIDIA k20c GPUs we reduce the runtime for the 150M×150M case to 45 seconds and the 450M×450M case to under 3 minutes, achieving a cross-match rate of over 1 trillion candidate pairs per millisecond; see Figure 2.

Our method has 3 main variables which control how the matching algorithm is run, each of which also effect the overall runtime. We have the h value, which controls the height of each zone (arcseconds), the θ value, which controls the search radius (arcseconds) and n, which controls the segment size. For our testing we set the search

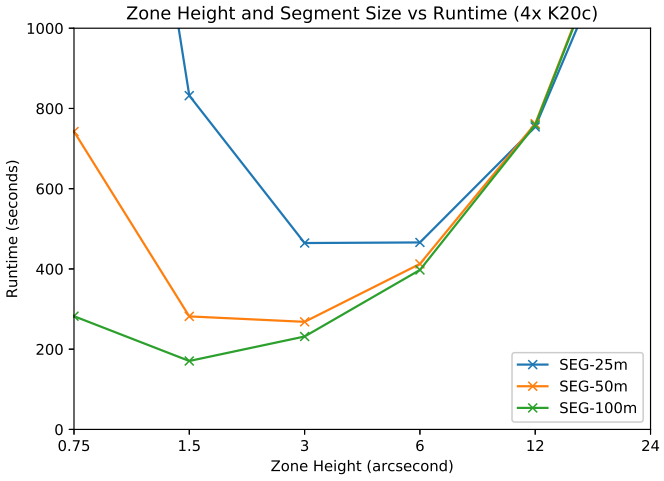

Figure 3: Runtime Performance as a function of the zone height and segment size. As the segment size increases to the maximum possible on an Nvidia k20c GPU (100 million), we approach best case performance. The smaller the number of total segments, the more time can be dedicated to the actual distance computations, therefore increasing the performance. Also note the effect of zone height, depending on the search radius, the zone height dictates how many matches are computed per zone-zone comparison. When the zone height is too large, we loose the advantage of limiting our search area and when the zone height is too small the overhead of launching extra kernels overtakes the advantage of limiting our search area.

```
Usage: ./BoostXmatch.x [options] file(s)
  Options:
    -o [ --outdir ] [=arg(=out)] Output directory path
    -m [ --maxout ] arg (=0)     Maximum output per job, default to (jobsize)^2
    -r [ --radius ] arg (=1.5)   Search radius in arcsec, default is 1.5"
    -z [ --zoneheight ] arg (=0) Zone height in arcsec, defaults to radius
    -t [ --threads ] arg (=0)    Number of threads, defaults to # of GPUs
    -n [ --numobj ] arg          Number of objects per segment
    -v [ --verbose ] [=arg(=9)]  Enable verbosity (implicit level 9=ALL)
    -x [ --overwrite ]           Overwrite output directory
    -h [ --help ]                Print help message
```

Usage 1: Help output for the BoostXmatch tool.

radius, to 1.5 arcseconds, which exceeds the realistic maximum angular object separation for SDSS, (Budavári and Szalay, 2008). The search radius obviously has a large impact on our search time, the larger the search radius the more objects need to be compared. The search radius should be large enough to account for measurement errors but not larger than necessary.

In order to achieve the fastest and most efficient matching given available hardware resources, we want to maximize our Segment size, which is limited by the total memory available on each GPU. For our tests on the k20c this number is approximately 100 million objects per segment. Finally we fix the zone height, which dictates how many sources will be compared for each zone-zone matching. For small values of h, the majority of the time is spent launching CUDA kernels, but large values of z increase the number of sources in each zone, forcing us to do more comparisons than necessary. Our testing concluded the optimal value for 4 GPUs is 100 million objects per segment and $h = 1.5$, for a two factor comparison of the effect of adjusting segment size and zone height, see Figure 3.

4. Conclusions

In this study we take a fresh look at efficient catalog matching, one of the biggest challenges of modern observational astronomy in the multi-wavelength and time-domain era. Building on ideas of the Zones Algorithm, we developed a novel approach to finding associations on the many-core general-purpose graphics processors in today's video cards. Our C++ and CUDA implementation achieves unprecedented speeds and can process two catalogs of 450 million entries each in less than 3 minutes using four NVIDIA Tesla K20c cards. Smaller catalogs and subsets of interest can be processed in interactive times with these new tools which means that on-the-fly data fusion in services such as SkyQuery can scale to the next generation of time-domain surveys.

An easy to use Python interface as well as a Python helper script being tested currently and available on Github along side the current C++/CUDA implementation. Future works includes tuning of the thread layout in accordance to newer compute capabilities to better utilize latest GPU architectures, dynamic zone sizing to minimize zones with low numbers of comparisons as well as the integration of this engine into the SkyQuery cluster solution.

Acknowledgments

This study was partially supported by funding from the NSF via grant AST-1412566 and NASA via the awards NNG16PJ23C and STScI-49721 under NAS5-26555.

References

N. Bell and J. Hoberock. Thrust: A productivity-oriented library for cuda. *GPU computing gems Jade edition*, 2:359–371, 2011.

T. Boch, F. X. Pineau, and S. Derriere. CDS xMatch Service Documentation, May 2016.

T. Budavari and M. A. Lee. Xmatch: GPU Enhanced Astronomic Catalog Cross-Matching. Astrophysics Source Code Library, Mar. 2013.

T. Budavári and T. J. Loredo. Probabilistic record linkage in astronomy: Directional cross-identification and beyond. *Annual Review of Statistics and Its Application*, 2(1):113–139, 2015. doi: 10.1146/annurev-statistics-010814-020231. URL `http://dx.doi.org/10.1146/annurev-statistics-010814-020231`.

T. Budavári and A. S. Szalay. Probabilistic cross-identification of astronomical sources. *The Astrophysical Journal*, 679(1):301, 2008.

T. Budavári, L. Dobos, and A. S. Szalay. SkyQuery: Federating astronomy archives. *Computing in Science & Engineering*, 15(3):12–20, May 2013. ISSN 1521-9615. doi: 10.1109/MCSE.2013.41. URL `http://scitation.aip.org/content/aip/journal/cise/15/3/10.1109/MCSE.2013.41`.

R. G. Crittenden. Igloo Pixelizations of the Sky. *Astrophysical Letters and Communications*, 37:377, 2000.

L. Dobos, T. Budavári, N. Li, A. S. Szalay, and I. Csabai. Skyquery: an implementation of a parallel probabilistic join engine for cross-identification of multiple astronomical databases. In *International Conference on Scientific and Statistical Database Management*, pages 159–167. Springer, 2012.

K. M. Górski, E. Hivon, A. J. Banday, B. D. Wandelt, F. K. Hansen, M. Reinecke, and M. Bartelmann. HEALPix: A Framework for High-Resolution Discretization and Fast Analysis of Data Distributed on the Sphere. *Astrophysical Journal*, 622:759–771, April 2005. doi: 10.1086/427976.

J. Gray, M. Nieto-Santisteban, and A. Szalay. The zones algorithm for finding points-near-a-point or cross-matching spatial datasets. *arXiv preprint cs/0701171*, 2007.

M. Harris, S. Sengupta, and J. Owens. Parallel prefix sum (scan) with cuda. *GPU Gems*, 3(39):851–876, 2007.

S. Heinis, T. Budavári, and A. S. Szalay. Cross-identification performance from simulated detections: Galex and sdss. *The Astrophysical Journal*, 705(1):739, 2009.

G. Kerekes, T. Budavári, I. Csabai, A. J. Connolly, and A. S. Szalay. Cross Identification of Stars with Unknown Proper Motions. *Astrophysical Journal*, 719:59–66, August 2010. doi: 10.1088/0004-637X/719/1/59.

P. Z. Kunszt, A. S. Szalay, and A. R. Thakar. The Hierarchical Triangular Mesh. In A. J. Banday, S. Zaroubi, and M. Bartelmann, editors, *Mining the Sky*, page 631, 2001. doi: 10.1007/10849171_83.

C. Nvidia. C programming guide version 4.0. *Nvidia Corporation*, 2011.

R. Scranton, M. Tegmark, and Y. Xu. URL `http://lahmu.phyast.pitt.edu/scranton/SDSSPix`.